ConstructTrack

Viva Preparation Guide

What the system does · Complete tech stack · Core concepts
Likely viva questions with model answers · Demo script · Key numbers

Team 54 — Ashish Upadhyay · Yash Rajput · Sahil Rao
Faculty Guide: Dr. Seema Kalonia
Minor Project (ES-451) · Department of IT · MAIT, Delhi · October 2026
Live: https://construct-track-pi.vercel.app

1. The project in one minute

> **30-second pitch:** "ConstructTrack is a web system for construction project managers. They record planned versus actual progress and upload site photos. A YOLO11 model detects objects in the photos as supporting evidence. From the project's own progress history, our engine predicts the completion date, the delay in days and a 0–100 schedule-risk score, and every number is explainable. Everything is compiled into an automatic report with a PDF export."

The problem. On most sites, progress lives in spreadsheets and photos sit in WhatsApp groups. Nobody combines them regularly, so slippage is noticed late and every stakeholder report is assembled by hand.

Our idea. Build decision support from data every project already has — planned % and actual % at known dates — and make every number transparent and reproducible. Computer vision is shown next to the numbers as evidence, but it never silently changes them.

Three things that make it different:

- **Explainable risk score:** four factors with fixed, published weights instead of a black-box ML model.
- **Data-sufficiency awareness:** with fewer than two usable records the system refuses to predict instead of guessing.
- **AI kept separate:** YOLO11 detections are evidence only, never an input to progress or risk numbers, because a general COCO model cannot measure construction completion.

2. What the system does (user's view)

Step	What the user does	What the system does
1. Create project	Enters name, location, start date, expected end date	Stores the project; dashboard counts update
2. Record progress	Adds planned % and actual % for a date	Computes deviation = actual − planned and a status: AHEAD / ON_TRACK / AT_RISK / DELAYED
3. Upload photos	Uploads JPEG/PNG/WEBP site photos, optionally linked to a progress record	Shrinks big photos in the browser, checks the real file type on the server, stores them in a private bucket
4. Analyse photo	Clicks Analyse	Sends a 1-hour signed URL to the AI service; YOLO11 returns objects, counts, confidence and an annotated image; results are saved
5. Progress analytics	Opens project page	Shows KPIs, planned-vs-actual chart, real elapsed-time trend, AI detection trend
6. Delay prediction	Scrolls to Delay Prediction	Projects completion date, delay in days, delay-probability heuristic
7. Risk analysis	Reads the risk card	0–100 score with LOW / MODERATE / HIGH / CRITICAL band and per-factor explanations
8. Generate report	Clicks Generate report	Builds a 9-section snapshot from the database and saves it

Step	What the user does	What the system does
9. PDF	Clicks Download PDF	Server renders the same snapshot as a PDF with PDFKit

3. Architecture

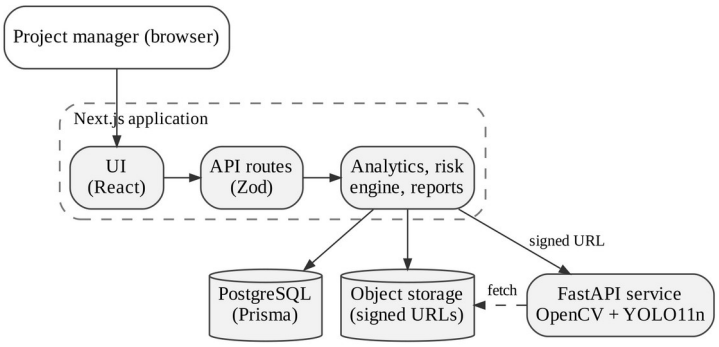


Figure 1: System architecture

Two independent services. The **Next.js app** (on Vercel) holds the UI, API routes, database access and storage access. The **FastAPI AI service** only does image preprocessing and YOLO11 inference; it receives a short-lived signed image URL and never holds database or storage credentials.

3.1 What happens when you click "Analyse"

1. Browser calls POST /api/analysis with the image ID.
2. API checks the image exists and is not already PROCESSING (otherwise returns 409 Conflict), then marks it PROCESSING.
3. Server creates a signed Supabase URL valid for 1 hour.
4. Server calls the AI service: POST /api/v1/analyze { image_url } (with the X-API-Key header if configured).
5. AI service downloads the image (size-capped, magic-byte check), decodes with OpenCV, resizes to ≤ 1280 px, runs YOLO11n (confidence 0.25, IoU 0.45), counts objects, draws boxes.
6. Next.js saves the AIAnalysis row and the annotated image, marks the image COMPLETED (or FAILED), and the UI shows results.

3.2 What happens when you generate a report

The client sends only a projectId. The server loads the project, progress records, images and analyses, runs the same analytics and risk functions used on screen, and saves the composed result as a JSON snapshot (Report.reportData). The web view and the PDF both render that snapshot, so they always match.

4. Complete technology stack

Layer	Technology	Role in ConstructTrack	Why we chose it
Frontend	Next.js 15 (App Router), React 19	Pages, layouts, server and client components	Full-stack in one codebase; server rendering; first-class Vercel hosting
Language	TypeScript	All web code	Type safety catches bugs at compile time
Styling / UI	Tailwind CSS 4, shadcn/ui (Radix UI)	Layout, cards, dialogs, tables	Fast, consistent, accessible components
Charts	Recharts	Planned-vs-actual and detection-trend charts	Simple React chart library
API	Next.js API routes	REST endpoints for projects, progress, images, analysis, reports	No separate backend server needed
Validation	Zod	Validates every request body/query	Rejects bad input with clear errors; types inferred from schemas
Database	PostgreSQL (Supabase)	Projects, records, images, analyses, reports	Relational data with foreign keys; reliable; free tier
ORM	Prisma 6	Schema, type-safe queries, migrations	Schema as code; generated types; versioned migrations
File storage	Supabase Storage	Site photos and annotated images	Private buckets + signed URLs; same platform as DB
AI service	Python, FastAPI, Uvicorn, Pydantic	HTTP API around the model	Fast, typed, auto-generated docs at /docs
Computer vision	Ultralytics YOLO11n, PyTorch, OpenCV, NumPy	Object detection, decode/resize, annotation	State-of-the-art real-time detector; nano = fast on CPU
Reporting	PDFKit	Server-side PDF generation	Real PDF from data, not a browser screenshot
Testing	node:test via tsx (61 tests), pytest (32 tests), ESLint, tsc	Unit and API tests	Engines are pure functions, easy to test
Deployment	Vercel + Supabase; AI service in Docker / self-hosted	Live web app and DB; AI runs on our machine	Free tiers; Hugging Face made Docker Spaces paid in 2026

5. Core concepts — explained simply

5.1 Next.js and the web layer

- **App Router:** each folder under app/ is a URL. app/(dashboard)/projects/[id]/page.tsx is the project page; [id] is a dynamic segment.
- **Server vs client components:** server components run on the server and can read the database directly (fast, secure). Client components ("use client") run in the browser for interactivity — dialogs, uploads, charts.
- **API routes:** files like app/api/projects/route.ts export GET/POST functions. On Vercel each becomes a **serverless function** — it starts on demand and scales automatically.

- **Consistent error envelope:** every API returns { success: true, data } or { success: false, error: { code, message } }, so the UI handles errors uniformly and internals never leak.
- **Zod:** schema-based validation, e.g. planned progress must be a number from 0 to 100; invalid input returns 400 with a clear message.

5.2 Database, Prisma and migrations

Model	Holds	Key relations
User	Project manager (owner)	1 user → many projects
Project	Name, location, status, startDate, expectedEndDate	Has progress records, images, reports (cascade delete)
ProgressRecord	recordDate, plannedProgress, actualProgress, deviation, status	Unique per (project, date)
ConstructionImage	storagePath, filename, size, analysisStatus	Optional link to a ProgressRecord
AIAnalysis	model, detections, objectCounts, averageConfidence, annotated image path	One-to-one with an image
DelayPrediction	Per-record baseline estimate and risk level	One-to-one with a progress record
Report	reportData JSON snapshot	Belongs to a project

- **ORM** (Object-Relational Mapper): we write prisma.project.findMany() instead of raw SQL; Prisma generates TypeScript types from schema.prisma.
- **Migration:** a versioned SQL file that changes the schema. On Vercel, prisma migrate deploy applies pending migrations during the build, which is how our tables were created in Supabase automatically.
- **Cascade delete:** deleting a project deletes its records, images and reports, so no orphan rows remain.
- **Connection pooling:** serverless functions open many short connections, so we use Supabase's Session pooler URL (IPv4-compatible, works with migrations).

5.3 Storage and security concepts

- **Private bucket:** photos are not publicly readable — site photos can be sensitive.
- **Signed URL:** a temporary link (1 hour here) that grants access to one file. The AI service gets only this, never a key.
- **Anon/publishable key vs service-role/secret key:** the publishable key is safe in the browser; the secret key bypasses security rules and lives only in server environment variables.
- **Magic-byte sniffing:** we check the first bytes of the file (JPEG starts FF D8 FF, PNG starts 89 50 4E 47) instead of trusting the filename or Content-Type.
- **Server-computed metrics:** the client can never send a progress %, risk score or prediction; the server always computes them from the database.

5.4 Computer vision concepts

- **Object detection:** find objects in an image and return, for each one, a class (e.g. person, truck), a bounding box (x1, y1, x2, y2) and a confidence score (0–1).
- **CNN:** a convolutional neural network learns visual features (edges → shapes → objects) through stacked convolution layers.
- **YOLO ("You Only Look Once"):** a single-stage detector — one forward pass predicts all boxes and classes, so it is fast enough for real time. Two-stage detectors (e.g. Faster R-CNN) first propose regions, then classify them — more accurate on small objects but slower.
- **YOLO11n:** Ultralytics' 2024 YOLO version, "n" = nano (smallest, fastest, about 2.6 M parameters) — suited to CPU inference.
- **COCO:** the 80-class dataset the model was pretrained on (person, car, truck, bicycle, …). It has no construction classes like "slab" or "brick wall" — the reason we treat detections as evidence only.
- **Confidence threshold (0.25):** detections below 25% confidence are dropped.
- **IoU (Intersection over Union):** overlap of two boxes = area of intersection ÷ area of union.
- **NMS (Non-Maximum Suppression):** when several boxes cover the same object, keep the highest-confidence one and remove others whose IoU with it exceeds 0.45.
- **Preprocessing:** OpenCV decodes the bytes and resizes so the longest side ≤ 1280 px (no distortion); YOLO letterboxes internally.
- **Model adapter pattern:** detector code sits behind a ModelAdapter interface, so a construction-specific model can replace YOLO11 without touching the API.

5.5 Progress analytics formulas

```
deviation = actualProgress − plannedProgress
AHEAD ≥ +3 · AT_RISK ≤ −3 · DELAYED ≤ −10 · else ON_TRACK
weekly rate = (latest actual − earliest actual) / real weeks elapsed
```

We use real calendar time between records, not an assumed weekly cadence, so irregular reporting does not distort the rate. With fewer than two records that carry an actual value, the trend is shown as unavailable.

5.6 Delay prediction and the risk engine

```
v  = (A_n − A_1) / (t_n − t_1)   // observed velocity (% per day)
v* = (100 − A_n) / (T − t_n)   // required velocity to finish on time
ρ  = v / v*   // velocity ratio
t̂  = t_n + (100 − A_n) / v   // projected completion (only if v > 0)
δ  = max(0, t̂ − T)   // predicted delay in days
R  = 0.30·S_V + 0.30·S_ρ + 0.25·S_D + 0.15·S_T   // risk score (0–100)
```

Factor	Weight	Scoring
Schedule variance S_V	30%	Δ ≥ 0 → 10; [−5,0) → 40; [−15,−5) → 65; < −15 → 90
Velocity adequacy S_ρ	30%	ρ ≥ 1 → 10; [0.8,1) → 35; [0.5,0.8) → 65; < 0.5 → 90

Factor	Weight	Scoring
Predicted delay S_D	25%	Delay as share of planned duration; 30% or more → 100; early → 5
Historical trend S_T	15%	Change in deviation over last 3 records: improving 20 / stable 50 / worsening 80

Bands: LOW 0–24 · MODERATE 25–49 · HIGH 50–74 · CRITICAL 75–100. **Estimated Delay Probability** = 0.40·S_V + 0.40·S_ρ + 0.20·S_T — a heuristic, not a statistical probability. **Data sufficiency:** 0–1 usable records INSUFFICIENT (no prediction), 2 LIMITED, 3–4 ADEQUATE, 5+ STRONG.

5.7 Evaluation (what we proved, and what we did not)

Scenario	Deviation	Velocity ratio	Predicted delay	Our risk	Deviation-only baseline
Office, slipping	−10	0.67	70 days	72 HIGH	MEDIUM
Tower, on plan	−1	1.00	2 days	23 LOW	LOW
Retail, ahead	+6	1.33	21 days early	10 LOW	LOW
Annex, stalled	−21	0.30	321 days	91 CRITICAL	CRITICAL
Single record	−2	—	—	withheld	LOW

- **Determinism:** 1,000 repeated runs → identical output.
- **Sensitivity:** ±20% on any one weight → 0 band changes in 32 cases; 10,000 random ±30% weight sets → max score shift 5 points, ranking preserved 10,000/10,000; band changes 4.6%, only for the tower, whose score (23) sits next to the LOW/MODERATE edge.
- **Not proved:** forecast accuracy — that needs outcome data from completed projects (future work).
- Reproduce everything with: cd web && npm run evaluate

5.8 Deployment concepts

- **Vercel:** hosts the Next.js app; builds on every git push to main; API routes run as serverless functions. Root directory = web.
- **Build pipeline:** npm install → prisma generate → prisma migrate deploy → next build.
- **Environment variables:** secrets (DATABASE_URL, Supabase keys) live in Vercel settings, never in Git.
- **Serverless limits we handled:** 4.5 MB request body → photos are downscaled in the browser; longer timeouts (maxDuration 60 s) for AI analysis.
- **AI service hosting:** containerised with a Dockerfile (CPU PyTorch, model baked in). Free Docker hosting on Hugging Face became paid in 2026 and PyTorch is too large for Vercel, so we run the service on our own machine for demos; every analysis result is stored in the database, so it remains visible online at any time.

6. Likely viva questions with model answers

6.1 Problem and overview

Q1. What problem does your project solve?

Construction progress is tracked in scattered spreadsheets and photo folders, so delays are spotted late and reports are made by hand. ConstructTrack centralises progress data and photos, predicts delay and schedule risk from the project's own history, and generates reports automatically.

Q2. Who is the user?

A construction project manager or site engineer who records progress, uploads site photos, and needs to report status to owners or stakeholders.

Q3. What is novel compared to existing tools?

Most research uses either vision with BIM models or machine learning trained on many past projects. We give useful, explainable risk assessment from data every project already has, refuse to predict when data is insufficient, and keep AI output separate from metrics so a general model cannot distort the numbers.

Q4. Is this an AI project if the risk score isn't machine learning?

Yes, it combines a real deep-learning model (YOLO11 object detection) with a deterministic, explainable decision-support engine. Choosing rules over ML for risk was deliberate: we have no labelled outcome data to train on, and an untrained ML model would be less trustworthy than transparent formulas.

Q5. Explain the complete flow in one sentence.

Create a project, record planned and actual progress, upload and analyse site photos, then the engine computes analytics, projected completion and a risk score, and everything is compiled into a report and PDF.

6.2 Web architecture and code

Q6. Why Next.js instead of a separate React + Express backend?

Next.js gives frontend and backend (API routes) in one TypeScript codebase, server-side rendering for fast pages, and native deployment on Vercel. It reduces moving parts for a small team.

Q7. What is the difference between a server component and a client component?

Server components render on the server and can query the database directly without exposing anything to the browser. Client components run in the browser for interactivity such as forms, uploads and charts; they are marked with "use client".

Q8. Why is the AI part a separate Python service?

The best computer-vision libraries (PyTorch, Ultralytics, OpenCV) are in Python, and the model is too large for serverless functions. Separating it also means each service can be scaled, deployed or replaced independently — a microservice design.

Q9. How do the two services communicate?

Over HTTP with JSON. Next.js calls POST /api/v1/analyze with a short-lived signed image URL; the FastAPI service returns detections, counts, average confidence and an annotated image in base64.

Q10. How do you validate input?

With Zod schemas on every API route — for example, progress must be 0–100 and dates must be valid. Uploaded files are validated by real content (magic bytes) and size, not by filename.

Q11. What happens if two people click Analyse on the same image at once?

The API marks the image PROCESSING first; a second request while it is processing gets HTTP 409 Conflict, so the model never runs twice for the same image.

Q12. How do you handle errors?

All routes use one wrapper that returns a consistent JSON envelope with an error code and safe message. Internal details such as SQL errors or stack traces are logged on the server, never sent to the client.

Q13. What are REST APIs and which ones did you build?

REST maps resources to URLs and HTTP methods: GET reads, POST creates, PATCH updates, DELETE removes. We built routes for projects, progress, images, analysis, delay prediction and reports, including GET /api/reports/:id/pdf.

6.3 Database

Q14. Why PostgreSQL and not MongoDB?

Our data is relational — projects have records, images, analyses and reports with clear foreign keys and cascade rules. PostgreSQL enforces those relationships and supports JSON columns where we need flexible data, such as detections and report snapshots.

Q15. What is Prisma and why use it?

Prisma is a type-safe ORM. We define the schema once in schema.prisma; it generates TypeScript types and query methods, and manages versioned migrations, so the database and code never drift apart.

Q16. What is a migration?

A versioned SQL change to the database schema. Our initial migration creates all tables and enums; on Vercel, prisma migrate deploy applies it automatically during the build.

Q17. Why store the report as a JSON snapshot?

A report should show what the system said at that moment. Saving a snapshot keeps report history stable even if progress data changes later, and both the web view and PDF render the same snapshot.

Q18. How is a photo linked to progress?

ConstructionImage has an optional progressRecordId. At upload, the user can choose a progress record; the server checks the record belongs to the same project before saving.

6.4 Computer vision and YOLO

Q19. How does YOLO work?

It divides the image into a grid and, in a single forward pass of a convolutional network, predicts bounding boxes, class probabilities and confidence scores. Overlapping boxes are cleaned up with non-maximum suppression. One pass is why it is fast.

Q20. Why YOLO11n specifically?

It is the latest Ultralytics generation (2024) and the nano variant is the smallest and fastest — it runs comfortably on a CPU, which suits a student project without GPUs.

Q21. What is IoU and NMS?

IoU is the overlap between two boxes divided by their combined area. NMS keeps the highest-confidence box and removes other boxes for the same class whose IoU with it exceeds a threshold — 0.45 in our configuration.

Q22. What does the confidence threshold do?

Detections below 0.25 confidence are discarded to reduce false positives; raising it gives fewer but surer detections.

Q23. Can YOLO tell how complete a building is?

No. It is pretrained on COCO, which has 80 everyday classes such as person, car and truck, and no construction states. Using its counts as a completion percentage would be scientifically wrong, so we show detections only as supporting evidence, such as more workers or vehicles on site than last visit.

Q24. Then what is the AI actually useful for here?

It turns unstructured photos into structured, searchable evidence — counts of people, vehicles and equipment over time, with annotated images — which a manager reviews next to the progress numbers. It also establishes the pipeline into which a construction-specific model can be plugged later.

Q25. How would you make it construction-specific?

Collect and label site images (e.g. formwork, rebar, excavator, crane), fine-tune YOLO11 on them, measure precision and recall, and swap it in through our ModelAdapter interface without changing the API.

Q26. What is the role of OpenCV?

It decodes the downloaded image bytes, resizes the image so the longest side is at most 1280 px without distortion, and draws the bounding boxes and labels for the annotated output.

Q27. What are precision and recall?

Precision = correct detections ÷ all detections (how many we flagged were right). Recall = correct detections ÷ all real objects (how many real ones we found). mAP averages precision across recall levels and classes; we have not measured these on construction data — that is future work.

6.5 Delay prediction and risk engine

Q28. How do you predict the completion date?

We measure the actual progress rate over real elapsed time between the first and latest records, then divide the remaining percentage by that rate and add it to the latest record date. If the rate is zero or negative, we give no date, because a stalled project has no finite completion date.

Q29. Explain the risk score formula.

R = 0.30 × variance score + 0.30 × velocity score + 0.25 × predicted-delay score + 0.15 × trend score. Each factor is scored 0–100 from fixed thresholds, and the result is banded LOW, MODERATE, HIGH or CRITICAL.

Q30. Why these weights?

Current variance and pace are the strongest signals of where the project is heading, so they get 30% each; the projected delay summarises their combined effect (25%); the trend adds direction but is noisier with few records (15%). We tested sensitivity: changing weights by up to ±30% never changed the ranking of our scenarios.

Q31. Why not use machine learning for delay prediction?

ML needs many completed projects with known outcomes; we have none. A model trained on no real data would give confident but meaningless numbers. Our engine is transparent, testable and honest; once outcome data exists, the weights can be calibrated or replaced with a fitted model.

Q32. Why is 'Estimated Delay Probability' not a real probability?

A true probability must be calibrated against real outcomes. Ours is a weighted heuristic of the same factors, so we label it explicitly as a heuristic in the UI and reports.

Q33. What happens with only one progress record?

Velocity needs at least two points in time, so the engine returns INSUFFICIENT_DATA and shows no risk score or date — it refuses to guess.

Q34. Why did the slipping office get HIGH while the baseline said MEDIUM?

Its deviation is −10, which alone looks moderate, but its pace is 2 %/week against 3 %/week needed (ratio 0.67) and the gap is widening every record. Combining factors projects a 70-day overrun, which is why it is HIGH.

Q35. Is the engine deterministic?

Yes — it reads only stored dates and progress values, never the current clock, so the same input always gives the same output. We verified this over 1,000 repeated runs and with unit tests.

6.6 Evaluation and testing

Q36. How did you evaluate the system?

On structured scenarios (slipping, on-plan, ahead, stalled, and two edge cases) against a deviation-only baseline, plus a determinism check and a weight-sensitivity analysis; all reproducible with npm run evaluate. Software is covered by 61 web unit tests and 32 AI-service tests.

Q37. Why didn't you report accuracy?

Accuracy needs ground truth — actual completion dates of real projects for prediction, and labelled construction images for detection. We do not have those yet, so we report what we can prove honestly: consistency, discrimination, reproducibility and robustness.

Q38. What does the sensitivity analysis show?

That conclusions don't hinge on exact weights: ±20% on any single weight changed no band, and 10,000 random ±30% weight sets moved scores by at most 5 points and never changed the project ranking.

Q39. What kind of tests did you write?

Unit tests for pure functions — progress analytics, delay prediction, risk scoring, report composition and PDF generation — and pytest tests for preprocessing, aggregation, annotation, the API schema, health and analysis endpoints, including an end-to-end YOLO11 inference test.

6.7 Deployment and security

Q40. How is it deployed?

The Next.js app is on Vercel, which rebuilds on every push to GitHub; the build applies database migrations automatically. PostgreSQL and photo storage are on Supabase. The AI service is containerised with Docker and currently runs on our own machine.

Q41. Why isn't the AI service on the cloud too?

PyTorch and YOLO are too large for Vercel's serverless functions, and Hugging Face moved Docker Spaces to paid plans in 2026. Our Dockerfile runs unchanged on any Docker host, so cloud hosting is a configuration step, not a code change. Meanwhile every analysis result is stored in the database, so earlier analyses are always visible online.

Q42. How do you keep secrets safe?

Keys live only in environment variables on the server (Vercel settings and a git-ignored .env). The browser only gets the publishable key; the secret key and database URL never leave the server; the AI service receives only signed URLs and can require an API key.

Q43. What is a signed URL and why use it?

A temporary link that grants access to one private file for a limited time — one hour here. It lets the AI service download a photo without ever holding storage credentials.

Q44. How did you handle large phone photos on Vercel?

Vercel limits request bodies to about 4.5 MB, so the browser downscales images over 3.5 MB to a maximum of 2048 px JPEG before upload. The server still re-checks size and type.

6.8 Limitations and future scope

Q45. What are the main limitations?

No login or multi-user access yet; the risk weights are reasoned, not fitted to real outcomes; velocity is a two-point linear rate; YOLO11 is general-purpose and not benchmarked on construction images; and the PDF has no embedded charts.

Q46. What would you do next?

Add authentication and organisations — we have started this as BuildBridge, with roles and a tower → floor → stage model; collect outcome data to calibrate the weights; fine-tune a construction-specific detector; and add alerts when a project enters a higher risk band.

Q47. What is BuildBridge?

Our next iteration toward a real product: multi-tenant organisations with roles and invites, and a tower → floor → stage progress model that site engineers update from their phones. It is in progress and separate from this submission.

6.9 Tricky / follow-up questions

Q48. If YOLO doesn't affect the risk score, why include it at all?

Because managers need evidence, not just numbers. Photos plus detections let a reviewer verify claimed progress, and the pipeline is ready for a construction-trained model. Keeping it out of the score is a deliberate integrity decision.

Q49. Couldn't a user type fake progress numbers?

Yes — like any record-keeping system, it trusts recorded data. That is exactly why photo evidence is stored alongside each update; future work includes trust checks such as EXIF dates and duplicate-photo detection.

Q50. What if the expected end date has already passed?

Required velocity becomes undefined, the velocity factor is scored as high concern (90) if work remains, and the system still reports the projected date from the actual pace.

Q51. Did you use AI tools to write the code?

Answer honestly. A good, defensible answer: "We used AI coding assistants as tools, like most developers today, but we designed the architecture, reviewed and tested the code, and can explain every module and formula." Then prove it by walking through any file they point to.

Q52. Which part did each team member do?

Agree on this as a team before the viva and keep it consistent with your proforma. Each member should be able to explain their module end to end.

7. Three-minute demo script

1. **Dashboard (20 s):** "Portfolio view — 4 projects, average completion, projects at risk."
2. **Commercial Office Building (40 s):** show progress KPIs and the planned-vs-actual chart; point out the gap widening.
3. **Delay Prediction (40 s):** HIGH, score 72, projected about 70 days late; open the factor explanations and show the formula.
4. **Metro Station Annex (15 s):** CRITICAL — stalled site, the engine flags it.
5. **Photos and AI (40 s):** open an analysed image — detected objects, confidence, annotated image, and the "visual evidence only" label. If the AI service is running, analyse a new photo live.
6. **Report (25 s):** generate a report, open it, download the PDF.

> **Before you present:** open the live site once (it wakes the serverless functions), keep one already-analysed photo ready, and have the GitHub repo and paper PDF open in tabs.

8. Numbers to remember

Item	Value
Risk weights	30% variance · 30% velocity · 25% predicted delay · 15% trend
Risk bands	LOW 0–24 · MODERATE 25–49 · HIGH 50–74 · CRITICAL 75–100
Progress status cut-offs	AHEAD ≥ +3 · AT_RISK ≤ −3 · DELAYED ≤ −10
Data sufficiency	0–1 INSUFFICIENT · 2 LIMITED · 3–4 ADEQUATE · 5+ STRONG
YOLO settings	YOLO11n · confidence 0.25 · IoU 0.45 · longest side ≤ 1280 px · 80 COCO classes
Uploads	JPEG/PNG/WEBP · max 10 MB · browser downscale above 3.5 MB to ≤ 2048 px
Signed URL lifetime	1 hour
Tests	61 web unit tests · 32 AI-service tests
Evaluation	Office 72 HIGH vs baseline MEDIUM · 1,000 identical runs · 0/32 band changes at ±20% · ranking kept 10,000/10,000
Database	7 models: User, Project, ProgressRecord, ConstructionImage, AIAnalysis, DelayPrediction, Report
Live / code	construct-track-pi.vercel.app · github.com/yash06rajput/ConstructTrack